UNIVERSIDAD EAFIT

ST1630 · Sistemas Intensivos en Datos

Proyecto Final · Entregable 1

FraudRadar

Detección de fraude casi en tiempo real para una billetera digital

Diagnóstico, requisitos y arquitectura

Dominio	Fintech colombiana: pagos y transferencias P2P
Integrante 1	Mateo García Carreño - 1000409635
Integrante 2	Juan José Gomez - 1000302983
Integrante 3	Juan José Vargas - 1000313560
Integrante 4	Luis Moreno Gutierrez - 1000302522
Plataforma	AWS Academy Learner Lab

*La empresa del caso, **"PagaYa"**, es una billetera digital ficticia inspirada en productos como Nequi o Daviplata. Los volúmenes son supuestos del equipo, no cifras reales de esas empresas.*

Profesor: Andrés Sacre Alzate · Medellín, 2026

1. Definición del problema

Los analistas de riesgo de PagaYa necesitan detectar patrones de fraude en menos de 2 minutos desde que ocurre la transacción sospechosa. Hoy revisan reportes diarios, así que una cuenta comprometida puede vaciarse horas antes de que alguien lo note.

Contexto de negocio

PagaYa procesa pagos a comercios, transferencias P2P y retiros en corresponsales. El fraude más común en billeteras digitales tiene una firma temporal: muchas transacciones pequeñas en pocos minutos, montos muy por encima de lo habitual del usuario, o uso de la cuenta desde ciudades distantes en un intervalo imposible. Estos patrones solo se ven agregando eventos por usuario en ventanas de tiempo, no mirando cada transacción aislada.

El sistema **detecta y alerta**; no autoriza ni bloquea transacciones en línea. La decisión de bloquear la toma un analista.

Usuarios del sistema y decisiones que toman

Usuario	Qué consulta	Qué decide
Analista de riesgo (operación)	Alertas abiertas y el historial reciente del usuario alertado	Bloquear la cuenta, contactar al cliente o descartar la alerta
Jefe de riesgo / cumplimiento	Tendencias de alertas, montos en riesgo, tasa de falsos positivos por regla	Ajustar umbrales de las reglas y priorizar investigaciones
Equipo de producto	Canales, tipos de comercio y franjas horarias con más fraude	Agregar fricción (p. ej. doble factor) donde más se concentra el fraude

Preguntas que el sistema debe responder

Operacionales (camino caliente, menos de 2 minutos):

1. ¿Qué usuarios hicieron más de 5 transacciones en una ventana de 5 minutos?
2. ¿Qué usuarios movieron más de $2.000.000 COP en una ventana de 10 minutos?
3. ¿Qué transacciones superan 5 veces el monto promedio de los últimos 30 días del usuario?
4. ¿Qué cuentas operaron desde dos ciudades distintas en menos de 30 minutos?

Analíticas (capa Gold, actualizada cada 15 minutos):

5. ¿Cuántas alertas y cuánto monto en riesgo hubo por regla, ciudad y franja horaria en el último día, semana y mes?
6. ¿Qué canal (app, corresponsal, QR) y qué categoría de comercio concentran el mayor monto sospechoso?
7. ¿Qué porcentaje de alertas de cada regla fue confirmado como fraude por los analistas (tasa de falsos positivos)?

Por qué se necesita una arquitectura distribuida

Supuestos de escala de PagaYa: 8 millones de usuarios activos al mes y 12 millones de transacciones diarias en promedio (unas 140 por segundo). En quincenas y primas el tráfico sube hasta unas 3.000 transacciones por segundo.

- **Volumen:** 12 millones de eventos diarios suman unos 4.400 millones al año. Con eventos JSON de alrededor de 0,5 KB, la capa Bronze crece cerca de 2 TB por año.
- **Velocidad:** las reglas exigen agregaciones por usuario en ventanas de minutos sobre un flujo continuo. Una base relacional tendría que hacer consultas periódicas (polling) sobre la tabla transaccional, compitiendo con la operación del core.
- **Variedad:** conviven eventos transaccionales (JSON en streaming), perfiles de usuario con campos que varían según el nivel de verificación (documentos), y catálogos de comercios y ciudades (CSV por lotes).
- **Cargas mixtas:** el mismo dato se usa para alertas de baja latencia y para análisis históricos pesados. Separar el camino caliente (streaming + MongoDB) del camino frío (lakehouse) evita que uno degrade al otro.

Fuera del alcance

Modelos de machine learning, bloqueo automático de transacciones y datos reales de clientes. Los eventos se generan con un productor sintético que simula comportamiento normal e inyecta patrones de fraude conocidos.

2. Requisitos funcionales y no funcionales

El sistema tiene 12 requisitos funcionales, cada uno trazable a un componente del diagrama, y 7 no funcionales con valores medibles. Los valores de la columna "Verificación" son los que se medirán en la demo del E2.

2.1 Requisitos funcionales

ID	Requisito	Componente (diagrama)	Pregunta
RF-01	El productor debe generar transacciones sintéticas (pago QR, P2P, retiro) en JSON y publicarlas en el topic `transactions` de Kafka con `user_id` como llave, a una tasa configurable entre 10 y 1.000 eventos/s.	Productor Python → Kafka	Todas
RF-02	El productor debe inyectar patrones de fraude etiquetados (ráfaga, monto atípico, viaje imposible) en una proporción configurable, por defecto 2 %.	Productor Python	1–4
RF-03	El sistema debe persistir cada evento crudo en `bronze.transactions` (Delta Lake en S3) junto con topic, partición, offset y hora de ingesta, en menos de 30 s.	Spark Streaming job 1 → Bronze	Linaje
RF-04	El sistema debe validar el esquema de cada evento, eliminar duplicados por `transaction_id` y enviar los registros inválidos a `silver.transactions_rejected`.	Spark Streaming job 2 → Silver	Calidad
RF-05	El sistema debe enriquecer cada transacción válida con el perfil del usuario y la categoría del comercio mediante un join, y escribirla en `silver.transactions`.	Spark Streaming job 2 → Silver	3, 6
RF-06	El sistema debe evaluar las cuatro reglas de fraude con ventanas de tiempo por usuario y generar una alerta por cada regla disparada.	Spark Streaming job 2 (windowing)	1–4
RF-07	El sistema debe escribir cada alerta en la colección `alerts` de MongoDB con estado `abierta`, la regla, la evidencia y los `transaction_id` que la dispararon.	MongoDB	1–4

ID	Requisito	Componente (diagrama)	Pregunta
RF-08	El sistema debe permitir a un analista consultar las alertas abiertas de un usuario y cambiar su estado a `confirmada` o `descartada`.	MongoDB (mongosh / Compass)	7
RF-09	El sistema debe recalcular cada día el perfil de cada usuario (monto promedio y número de transacciones de los últimos 30 días) en `silver.user_profile`.	Airflow → Spark batch	3
RF-10	El sistema debe construir cada 15 minutos el modelo dimensional de Gold a partir de Silver y del estado de las alertas en MongoDB.	Airflow → Spark batch → Gold	5–7
RF-11	El orquestador debe verificar la frescura de Silver antes de construir Gold y detener el DAG si Silver no recibió datos en los últimos 10 minutos.	Airflow	Calidad
RF-12	El sistema debe exponer las tablas de Gold para consultas SQL en Amazon Athena mediante el Glue Data Catalog.	Glue Data Catalog → Athena	5–7

2.2 Requisitos no funcionales

ID	Atributo	Requisito	Verificación
RNF-01	Escalabilidad	Diseño para 3.000 eventos/s en pico. El topic `transactions` tiene 6 particiones, así que el consumo escala hasta 6 tareas de Spark en paralelo agregando executors o brokers, sin cambiar código.	En el lab: 200 eventos/s sostenidos durante 10 min y un pico de 1.000 eventos/s por 60 s, con consumer lag que vuelve a menos de 30 s en máximo 2 min.
RNF-02	Disponibilidad (CAP)	Kafka y MongoDB se configuran como **CP**: ante una partición de red prefieren rechazar escrituras antes que perder o duplicar alertas. Es aceptable porque el sistema no está en el camino de autorización del pago: unos segundos sin alertas no detienen transacciones.	Configuración documentada: `acks=all`, `unclean.leader.election.enable=false`; MongoDB con `writeConcern: majority`.
RNF-03	Consistencia (garantía de entrega)	Exactly-once efectivo de extremo a extremo: ningún evento se cuenta dos veces en Silver, Gold ni en las alertas, aunque un job falle a mitad (detalle por tramo en la tabla 2.3).	Matar el job de Spark durante la demo y comprobar que los conteos de Silver y de `alerts` no cambian al reiniciar.
RNF-04	Latencia	p95 evento → alerta en MongoDB < 2 min. Evento → Bronze < 30 s. Evento → Gold ≤ 20 min (DAG cada 15 min + hasta 5 min de ejecución).	Diferencia entre `event_ts` y `created_at` de las alertas; timestamp de la última versión Delta de Gold.
RNF-05	Tolerancia a fallos	Si cae un job de Spark, se reinicia solo (`restart: always`) y retoma desde su checkpoint en S3 sin pérdida, porque Kafka retiene los eventos 72 h. Si cae Kafka, el productor reintenta hasta 2 min (`delivery.timeout.ms`). Si cae MongoDB, el micro-batch falla y se reprocesa al reiniciar. Las tareas de Airflow tienen 2 reintentos.	Tiempo de recuperación < 10 min tras reiniciar una instancia EC2, sin eventos perdidos ni duplicados.

ID	Atributo	Requisito	Verificación
RNF-06	Trazabilidad	Cada fila de Bronze guarda topic, partición y offset. Silver y Gold conservan `transaction_id` y el `run_id` de Airflow. Cada alerta guarda los `transaction_id` que la dispararon. Delta Lake permite auditar versiones con `DESCRIBE HISTORY` y time travel.	Dada una alerta, reconstruir su camino hasta el offset exacto en Kafka.
RNF-07	Costo	El pipeline cabe en el presupuesto del Learner Lab: máximo 3 instancias EC2 tamaño *large* y apagado fuera de las sesiones de trabajo.	Consumo de créditos revisado cada semana en el panel del lab.

2.3 Garantía de entrega por tramo del pipeline

Tramo	Garantía	Mecanismo
Productor → Kafka	At-least-once sin duplicados por reintento	Productor idempotente: `enable.idempotence=true`, `acks=all`.
Kafka → Bronze y Silver (Delta)	Exactly-once	Spark guarda los offsets en el checkpoint y Delta confirma cada micro-batch de forma transaccional e idempotente.
Silver → MongoDB (alertas)	At-least-once + escritura idempotente = exactly-once efectivo	`foreachBatch` con upsert por `alert_id` determinístico = hash(`user_id`, regla, inicio de ventana).
Silver → Gold	Exactly-once	`MERGE` de Delta por llave de negocio; re-ejecutar la misma corrida de Airflow no duplica filas.

3. Diagrama de arquitectura de datos por capas

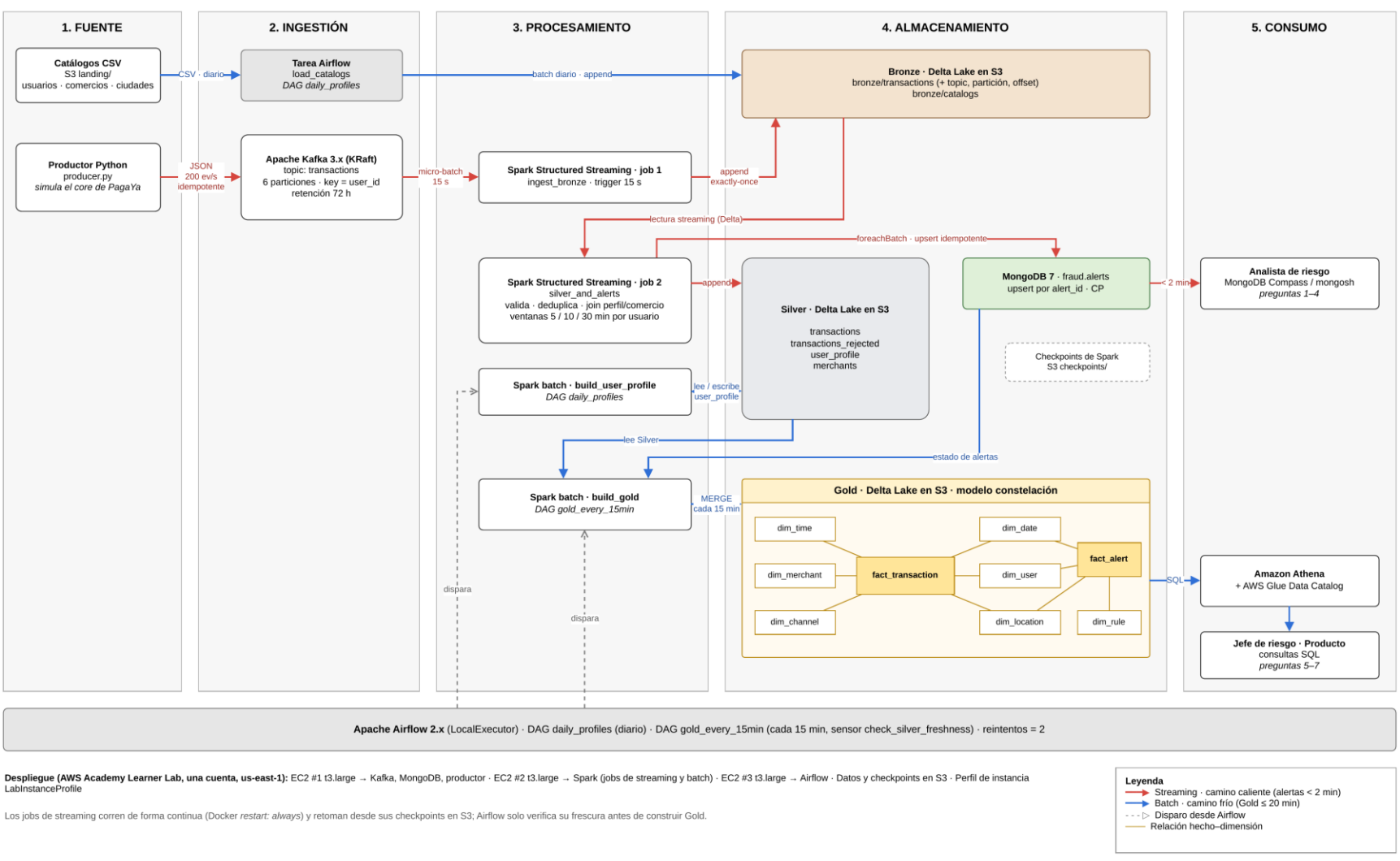


Figura 1. Arquitectura de datos por capas de FraudRadar.

El diagrama se lee de izquierda a derecha en cinco columnas (Fuente, Ingestión, Procesamiento, Almacenamiento, Consumo). Airflow es la franja inferior que dispara los procesos batch. Las tablas siguientes detallan cada caja y cada flecha.

4. Justificación de decisiones técnicas (ADRs)

Seis decisiones sostienen la arquitectura: Kafka con productor idempotente, Spark Structured Streaming, Delta Lake con modelo dimensional, MongoDB para alertas, Airflow y despliegue propio en EC2. Cada ADR sigue el formato visto en S10.

ADR-01. Apache Kafka como motor de ingestión en streaming

Contexto. RF-01 y RNF-01 exigen soportar picos de 3.000 eventos/s. RNF-05 exige poder reprocesar eventos tras una caída. Las reglas por ventana (RF-06) necesitan que los eventos de un mismo usuario lleguen en orden.

Alternativas consideradas.

- *Amazon Kinesis Data Streams:* gestionado, pero cobra por shard-hora aunque la sesión del lab esté apagada, y Spark de código abierto no trae un conector oficial de Kinesis para Structured Streaming.

Decisión. Kafka 3.x en modo KRaft, en Docker. Topic `transactions` con 6 particiones, llave `user_id` y retención de 72 h. Al usar `user_id` como llave, todos los eventos de un usuario caen en la misma partición y conservan su orden. El productor es idempotente (`enable.idempotence=true`, `acks=all`): los reintentos no crean duplicados en el log. En producción se usaría factor de replicación 3 con `min.insync.replicas=2` y sin elección de líder "sucia", una configuración **CP**.

Consecuencias. El equipo opera Kafka. En el lab hay un solo broker: es un punto único de falla, aceptado y documentado. Cambiar el número de particiones después rompe el mapeo llave → partición, así que las 6 particiones se fijan desde el inicio.

ADR-02. Spark Structured Streaming (micro-batch) para el procesamiento

Contexto. RNF-04 pide alertas en menos de 2 minutos, no en milisegundos. RF-06 requiere ventanas con estado por usuario. RF-09 y RF-10 son procesos batch que también necesitan un motor distribuido.

Alternativas consideradas.

- *Apache Flink:* procesamiento evento a evento con latencia de milisegundos y manejo de estado más fino. Se descarta porque la latencia requerida no lo exige y obligaría a operar un segundo motor, ya que Spark se necesita igual para el batch.

Decisión. Spark Structured Streaming con trigger de 15 s y watermark de 10 min para deduplicación y ventanas. Dos jobs: `ingest_bronze` (Kafka → Bronze) y `silver_and_alerts` (Bronze → Silver + alertas). Las alertas usan modo `update` con `foreachBatch`, para emitir en cuanto una ventana supera el umbral sin esperar a que cierre. La misma API de DataFrames sirve para streaming y batch.

Consecuencias. La latencia mínima es del orden del trigger (segundos). Los eventos que llegan con más de 10 min de retraso quedan en Bronze pero no entran a las ventanas. En modo `update`

una misma ventana puede emitirse varias veces, por lo que la escritura de alertas debe ser idempotente (ADR-04).

ADR-03. Delta Lake en S3 y modelo dimensional en Gold

Contexto. RNF-03 exige exactly-once hacia el almacenamiento y RNF-06 exige auditar versiones. Las preguntas 5 a 7 son agregaciones por fecha, regla, ciudad, canal y comercio.

Alternativas consideradas.

- *Apache Iceberg:* equivalente en transacciones ACID y time travel, con mejor soporte multi-motor. Se descarta porque el curso trabajó Delta (S4) y su integración con Spark (`MERGE`, fuente y destino de streaming) es más directa.
- *Tabla ancha única en Gold:* más simple de consultar, pero repite atributos de usuario y comercio en millones de filas y no permite dos granos distintos (transacción y alerta).

Decisión. Delta Lake en S3 para las tres capas. Gold es una constelación con `fact_transaction` y `fact_alert` que comparten dimensiones, cargada con `MERGE` por llave de negocio. `fact_transaction` se particiona por fecha. Las tablas se registran en el Glue Data Catalog y se consultan con Athena, que lee tablas Delta de forma nativa.

Consecuencias. Mantener dimensiones y llaves sustitutas agrega código en `build_gold`. Las dimensiones son SCD tipo 1, así que no se conserva el historial de cambios (por ejemplo, del nivel KYC). Los micro-batches generan archivos pequeños, por lo que se requiere `OPTIMIZE` diario. Athena solo lee Delta; todas las escrituras pasan por Spark.

ADR-04. MongoDB para las alertas (componente libre)

Contexto. RF-07 y RF-08: el analista consulta las alertas de un usuario en milisegundos y cambia su estado. Cada regla produce evidencia con estructura distinta: la ráfaga guarda conteo y ventana; el viaje imposible, dos ciudades y la distancia; el monto atípico, el promedio y el factor. Las alertas deben escribirse con upsert idempotente.

Alternativas consideradas.

- *Cassandra:* sistema AP con consistencia ajustable y escritura masiva excelente, pero el volumen de alertas (cerca del 2 % de los eventos) no necesita ese throughput, y su modelado rígido por consulta dificulta las consultas ad hoc del analista.
- *DynamoDB:* gestionado, pero sus consultas se limitan a la llave y a los índices definidos, y el equipo tiene más experiencia con MongoDB.

Decisión. MongoDB 7, colección `fraud.alerts`. El `_id` es un `alert_id` determinístico, hash de (`user_id`, regla, inicio de ventana), así que reescribir la misma alerta la actualiza en vez de duplicarla. Índice compuesto `{user_id: 1, status: 1, created_at: -1}`. Escritura y lectura con `majority`: posición **CP**. Si se fragmentara, la shard key sería `user_id` con hash: reparte las escrituras de forma uniforme y la consulta principal (alertas de un usuario) va a un solo shard. No se usa `created_at` porque es monotónico y concentraría todas las escrituras en el último shard.

Consecuencias. Ante una partición, un primario sin mayoría deja de aceptar escrituras: las alertas se retrasan y el micro-batch se reintenta, pero no se pierden. En el lab MongoDB corre en un solo nodo, sin réplica. El estado de las alertas vive en MongoDB y en Gold; Gold puede estar hasta 15 min desactualizado respecto a las decisiones del analista.

ADR-05. Apache Airflow para la orquestación

Contexto. RF-09 a RF-11: los procesos batch deben dispararse solos y en orden, y fallar limpiamente si algo anterior falla. Los jobs de streaming corren de forma continua.

Alternativas consideradas.

- *Cron en EC2:* sin dependencias, reintentos ni visibilidad del estado.
- *AWS Step Functions:* gestionado, pero se modela en JSON (ASL), no se vio en el curso.

Decisión. Airflow 2.x en Docker con LocalExecutor y dos DAGs:

- `gold_every_15min`: `check_silver_freshness` (sensor: última versión de `silver.transactions` con menos de 10 min) → `extract_alert_status` desde MongoDB y `build_dimensions`, en paralelo → `build_facts` (`MERGE`) → `data_quality_checks`.
- `daily_profiles`: `load_catalogs` → `build_user_profile` → `optimize_and_vacuum`.

Configuración común: 2 reintentos con 2 min de espera, `max_active_runs=1` para evitar dos `MERGE` simultáneos y `catchup=False`. Si el sensor falla, las tareas siguientes quedan en `upstream_failed` y Gold no se construye con datos incompletos.

Consecuencias. Airflow ocupa una instancia completa. El ciclo de vida de los jobs de streaming queda en Docker (`restart: always`); Airflow solo verifica su salud mediante el sensor. Con `max_active_runs=1`, una corrida lenta retrasa la siguiente.

ADR-06. Despliegue en EC2 con Docker Compose y datos en S3

Contexto. El Learner Lab limita EC2 a instancias hasta tamaño *large*, con máximo 32 vCPU simultáneas. Las sesiones duran 4 h y las instancias EC2 se apagan al terminar, mientras otros recursos siguen corriendo y consumiendo créditos. La guía exige un `docker-compose.yml` que levante el pipeline desde cero.

Alternativas consideradas.

- *Servicios gestionados (MSK, EMR, MWAA, Kinesis):* menos operación, pero cobran mientras existen aunque la sesión esté apagada, lo que pone en riesgo el presupuesto.
- *Todo local en portátiles:* sin costo, pero sin S3 ni Athena, y Kafka + Spark + Airflow + MongoDB necesitan más RAM de la que tiene un portátil típico.

Decisión. El mismo `docker-compose.yml` (con perfiles por servicio) corre en local para desarrollo y en 3 instancias t3.large en EC2. Tablas Delta y checkpoints van en S3, que persiste entre sesiones y cuesta poco. Glue y Athena son serverless y se pagan por consulta. Las instancias usan el perfil `LabInstanceProfile` en lugar de llaves en el código, porque las credenciales del lab rotan en cada sesión.

Consecuencias. El equipo opera cuatro sistemas y debe entenderlos todos para la defensa. No hay alta disponibilidad real (un broker, un nodo de MongoDB). Cada inicio de sesión es en la práctica una prueba de recuperación: los jobs deben retomar desde sus checkpoints en S3 (RNF-05).

5. Plan de implementación

El trabajo se reparte por componente del diagrama. Cada integrante es dueño de un componente y par de defensa de otro: revisa y aprueba sus PR, así cada parte del sistema la conocen al menos dos personas antes de la defensa individual.

5.1 Distribución de responsabilidades

Integrante	Componente	Entregables en el repositorio	Par de defensa
Mateo García	Infraestructura e ingestión	`docker-compose.yml` con perfiles; S3, EC2 y `LabInstanceProfile`; Kafka (KRaft, topic `transactions` de 6 particiones); `producer.py` con inyección de fraude; README.	Juan José Vargas
Juan José Gómez	Streaming Bronze / Silver	Job 1 `ingest_bronze`; job 2, parte Silver: validación de esquema, deduplicación con watermark, join con perfil y comercio, `transactions_rejected`.	Luis Moreno
Juan José Vargas	Reglas y alertas (camino caliente)	Ventanas de las 4 reglas; `foreachBatch` con upsert a `fraud.alerts`; índices; consultas del analista en mongosh; batch `build_user_profile` (insumo de la regla de monto atípico).	Mateo García
Luis Moreno	Gold y orquestación (camino frío)	`build_gold` (dimensiones y MERGE de los dos hechos); Glue + Athena con las queries de las preguntas 5 a 7; DAGs `gold_every_15min` y `daily_profiles`; `data_quality_checks`.	Juan José Gómez

5.2 Cronograma e hitos

Semana	Hito	Criterio de "hecho"
S13	H1 · Esqueleto y camino caliente mínimo	Contratos fijados (evento JSON, Silver, documento de alerta, tablas de Gold). `docker compose up` levanta Kafka, Spark, MongoDB y Airflow en local. Productor → Kafka → Bronze → Silver de punta a punta. Retroalimentación del E1 incorporada.
S14	H2 · Pipeline completo en local	Las 4 reglas escriben en MongoDB, `user_profile` se recalcula, Gold con los dos hechos es consultable en Athena y ambos DAGs terminan en verde. Prueba de matar el job sin duplicados (RNF-03). Asesoría con un problema concreto.
S15	H3 · Despliegue y pruebas	Pipeline corriendo en las 3 EC2. RNF-01, RNF-04 y RNF-05 medidos. Congelamiento de código, README, bitácora de IA, slides y ronda de preguntas cruzadas entre integrantes.
S16 − 1 día	Ensayo y PR final	Ensayo completo de la demo en EC2. PR final abierto antes de las 11:59 PM.
S16	Sustentación	Presentación (10 min), demo en vivo (8 min) y defensa (7 min).

5.3 Riesgos técnicos y mitigación

Riesgo técnico	Mitigación
Calendario comprimido: tres semanas de implementación antes de S16.	Trabajo en paralelo desde S13 sobre contratos fijos: las reglas y Gold se desarrollan contra una muestra fija de Silver mientras el streaming se completa. Si hay atraso, se recorta primero lo accesorio (OPTIMIZE, chequeos de calidad extra), nunca el flujo de punta a punta.

Riesgo técnico	Mitigación
Las sesiones de 4 h del Learner Lab apagan las EC2 y los recursos consumen créditos.	Desarrollo en local con el mismo compose. Tablas y checkpoints en S3. Script de arranque único. Revisión semanal de créditos.
Una t3.large (2 vCPU, 8 GB) no alcanza para 2 jobs de streaming más los batch.	Probar en una t3.large en S14. `spark.cores.max` y memoria acotados por job, `shuffle.partitions=6`. Si no alcanza, agregar un worker de Spark en la EC2 #1 o #3.
Crece el estado de las ventanas o se pierden eventos tardíos.	Watermark de 10 min y state store RocksDB. El productor inyecta eventos tardíos a propósito para probarlo.
Duplicados al reiniciar un job (se rompe el exactly-once).	`alert_id` determinístico y MERGE por llave de negocio. La prueba de matar el job se hace en S14, no por primera vez en la demo.
Athena no lee bien las tablas Delta registradas en Glue.	Validar en S13 con una tabla Delta de prueba. Plan B: manifiesto symlink o consultas con Spark SQL.
La demo falla en vivo.	Dataset precargado (permitido por la guía), runbook de arranque y ensayo obligatorio el día anterior.